

Memory-Centric Agentic Inference — cycles 44-45

2026-05-12

Contents

Memory-Centric Agentic Inference — cycles 44-45	1
Abstract	1
Introduction	2
Approach	2
Findings	3
Discussion	6
Open Questions	9
References	9
Appendix: Implementation Details	10

Memory-Centric Agentic Inference — cycles 44-45

Abstract

Cycles 44-45 extended the memory-centric architecture package from a validated memory-object control plane into a migration path for conventional traces. The prior architecture already had a memory-object application binary interface, or ABI, meaning a structured contract that tells the runtime, planner, and compiler what kind of memory object is being handled, what lifetime and security constraints apply, and whether placement, reuse, compression, migration, or retention actions are allowed.

Cycle 44 added M-TRACEABI-1, a legacy trace-to-ABI compatibility harness. It converts conventional request, runtime, and planner records into candidate memory-object contracts, then reruns those candidates through the existing ABI validator. The result is intentionally conservative: some prompt-prefix and retrieved-context candidates become ABI-admissible, some incomplete objects require annotation, unsafe or ambiguous cases fail closed, and opaque conventional requests remain executable as Option A.

Cycle 45 added M-ANNOT-1, a constrained annotation layer for fields that cannot be inferred safely from traces alone. The annotation layer can complete selected missing fields, such as reuse scope, tool provenance, verifier targets, branch parents, and durable-retention policy, but only after constraint checks and a second pass through the existing ABI validator. The cycle 45 auditor validated the work with no critical or moderate findings.

The combined result is a clearer control-plane migration story: conventional serving traces can be lifted into the memory-centric architecture only when they carry enough information, or when explicit constrained annotations complete the missing information. Rejected, conflicting, partial, or production-upgrade annotations emit zero downstream memory actions. All production calibration, readiness, threshold, causal-validity, and claim-credit fields remain false.

Introduction

The overall investigation asks whether future AI infrastructure should be organized around memory movement, placement, reuse, compression, and lifetime management, rather than arithmetic throughput alone. Earlier cycles built models, simulators, queueing analyses, security gates, production-evidence gates, and architecture options for this question. Public hardware and systems context remains the same: current accelerators and systems expose large but bounded HBM, host memory, interconnect, storage, and benchmark regimes [1]-[8], while serving systems already use memory-management techniques such as paged attention, prefix caching, semantic caching, CPU memory tiers, and CXL-style pooled memory [9]-[14].

The cycles reported here focus on the runtime and compiler control plane. In this report, “control plane” means the metadata, validation, and policy path that decides whether a memory object can be placed, reused, compressed, migrated, or retained. This is separate from the data plane, which actually stores and moves bytes.

The relevant architecture options are:

- **Option A:** conventional request/model/KV serving. It treats most memory state as opaque and remains the safe fallback.
- **Option B:** memory-object-aware runtime. It exposes typed memory objects, such as KV cache, prompt prefix, retrieved context, or tool output, to placement and reuse policy.
- **Option C:** trajectory/DAG memory fabric. It extends Option B to branch-aware and verifier-aware agent trajectories, where state has parent-child dependencies and uncertain future reuse.

Cycles 44-45 do not introduce production evidence. They extend the migration path from conventional traces into the validated Option B/C control plane while preserving Option A and keeping production claims blocked.

Approach

The source basis for this report is the supplied cycle range, supplied audit report, workspace artifacts, ledger rows, plan rows, validation outputs, figure files, and MANIFEST.md. No callable session-search or session-fetch tool was available in this environment, so source sessions are used as traceability anchors rather than directly quoted session transcripts.

Cycle	Milestone	Source sessions	Main evidence used
44	M-TRACEABI-1	researcher 15321c2d-5734-4434- 803e-bf9a4b7a5904; worker 5d05a8ba-a51c-460d- a13e-746f59f6584d; auditor 0c742462-3c65-4490- 815c-f53ea7d9aec3	promise_ ledger.jsonl, plan_of_record.md, memory-centric- agentic/trace_to_ abi_migration.md, trace-to-ABI CSVs and figures

Cycle	Milestone	Source sessions	Main evidence used
45	M-ANNOT-1	researcher 039107f3-6458-461e-ab7f-c10b11797eed; worker d6346464-fa09-46ff-a4a7-ee8e082f89db; auditor f0587bb3-2d2d-4b04-be7c-ded4019d58f0	supplied audit report, promise_ledger.jsonl, plan_of_record.md, memory-centric-agentic/memory_object_annotation_contract.md, annotation CSVs and figures

A gap in the record is explicit: full session histories were not fetchable. The report therefore relies on the supplied session IDs, the auditor validation summary, and the workspace artifacts.

Findings

Context Before Cycle 44: The ABI Was Already the Admission Boundary

Before these cycles, the architecture had already established a memory-object ABI and integration path. The ABI defines object classes, required fields, lifetime hints, security fields, and planner-admission rules. The integration harness then connects ABI-admitted objects to runtime and planner action outputs while preserving fail-closed behavior.

The relevant upstream rule is simple: a memory object may reach placement, reuse, compression, migration, or retention actions only after it passes the ABI validator. Rejected contracts emit zero downstream memory actions. Option A remains available for opaque conventional serving. Production calibration, readiness, threshold success, causal validity, and claim credit remain false for synthetic or internal control-plane evidence.

Cycles 44-45 did not replace this boundary. They added a migration path into it.

Cycle 44: Legacy Trace-to-ABI Migration

Cycle 44 added M-TRACEABI-1, a compatibility harness that lifts conventional trace, runtime, and planner records into memory-object ABI candidates. “Trace lifting” means taking information that already exists in logs or runtime decisions, such as request IDs, prompt IDs, runtime object IDs, tenant labels, retrieval IDs, branch IDs, or planner actions, and using it to construct a candidate ABI object.

The cycle note records an important input gap: the workspace did not contain data/synthetic_trace_v2.csv, data/synthetic_trace_v3.csv, or data/security_trace_v3.csv. The harness preserved that as trace_source=no_canonical_trace_csv_found and used validated runtime and planner artifacts plus explicit legacy fixtures to model the migration boundary.

The migration status classes were:

- `abi_admissible`: the lifted candidate passed the existing ABI validator and could enter the validated ABI integration boundary.
- `annotation_required`: the trace described a plausible object but lacked a mandatory field, such as reuse scope, verifier target, or retention policy.

- `fail_closed`: the candidate was unsafe or non-liftable as-is, such as missing provenance or dangling branch parents.
- `option_a_opaque_fallback`: no object contract was created, and conventional Option A serving remained executable.

The output `data/trace_to_abi_lift_results.csv` contains 9 rows: 3 ABI-admissible rows, 3 annotation-required rows, 2 fail-closed rows, and 1 Option A opaque fallback. The admissible rows showed that prompt-prefix and retrieved-context style objects can sometimes be lifted from conventional traces. The blocked rows showed that tool-output provenance, branch-parent resolution, verifier target integrity, durable retention policy, and ambiguous KV reuse scope cannot be safely invented.

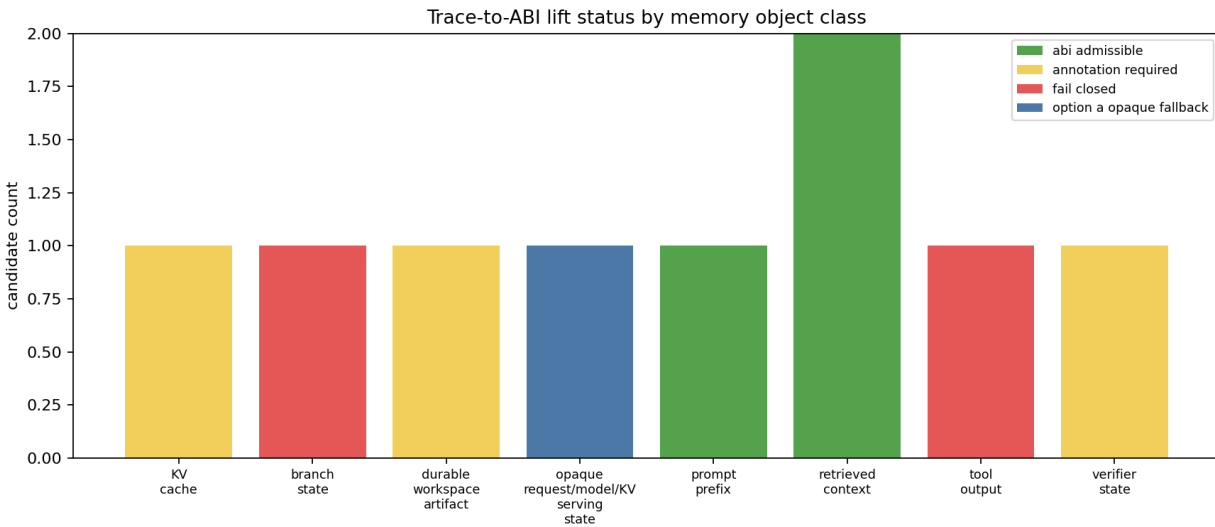


Figure 1: Trace-to-ABI lift status counts showing ABI-admissible, annotation-required, fail-closed, and Option A fallback outcomes.

The cycle 44 auditor validated the milestone after a scoped patch. The patch tied lifted admissible-candidate action counts to the existing ABI integration helper path. After that change, incomplete or ambiguous trace-derived contracts emitted zero downstream memory actions, Option A remained opaque, and synthetic/internal lifted traces kept all production-credit fields false.

The main decision from cycle 44 was that trace migration must be treated as compatibility, not authority. Conventional traces can propose memory-object contracts, but the ABI validator still decides whether those contracts are actionable.

Cycle 45: Annotation Contract for Non-Inferable Fields

Cycle 45 added `M-ANNOT-1`, a developer/runtime annotation contract for fields that trace lifting exposed but could not infer safely. An annotation is an explicit structured statement from a developer, runtime, compiler, security layer, or planner about a missing ABI field. The important constraint is that annotations are not trusted blindly. They are checked against trace, security, provenance, branch, verifier, retention, and evidence-label context.

The schema in `data/memory_object_annotation_schema.csv` defines eight annotation classes:

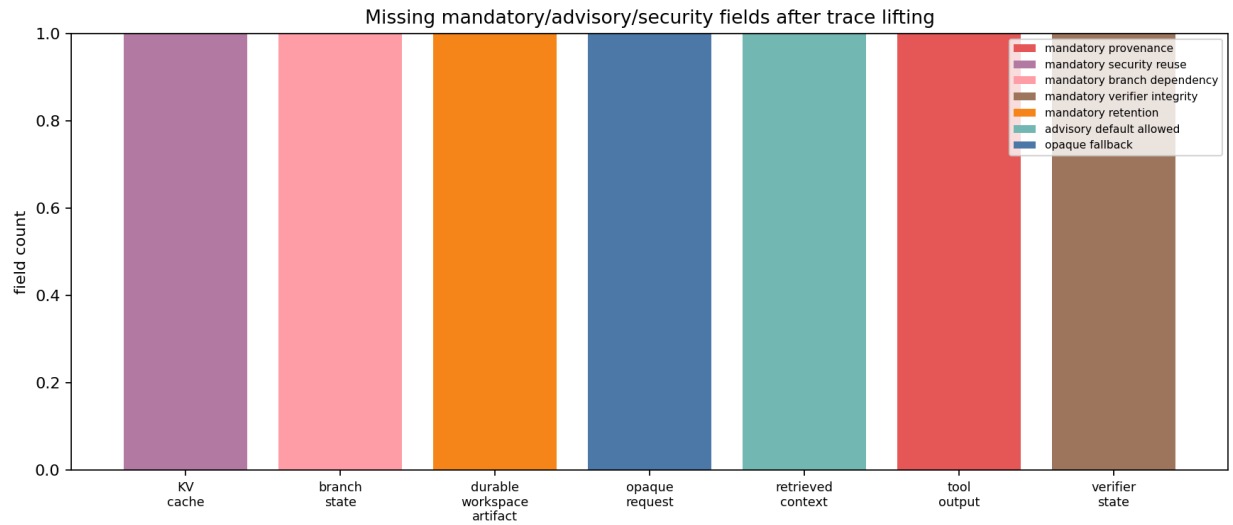


Figure 2: Missing-field analysis showing which conventional trace gaps require annotations or block ABI lifting.

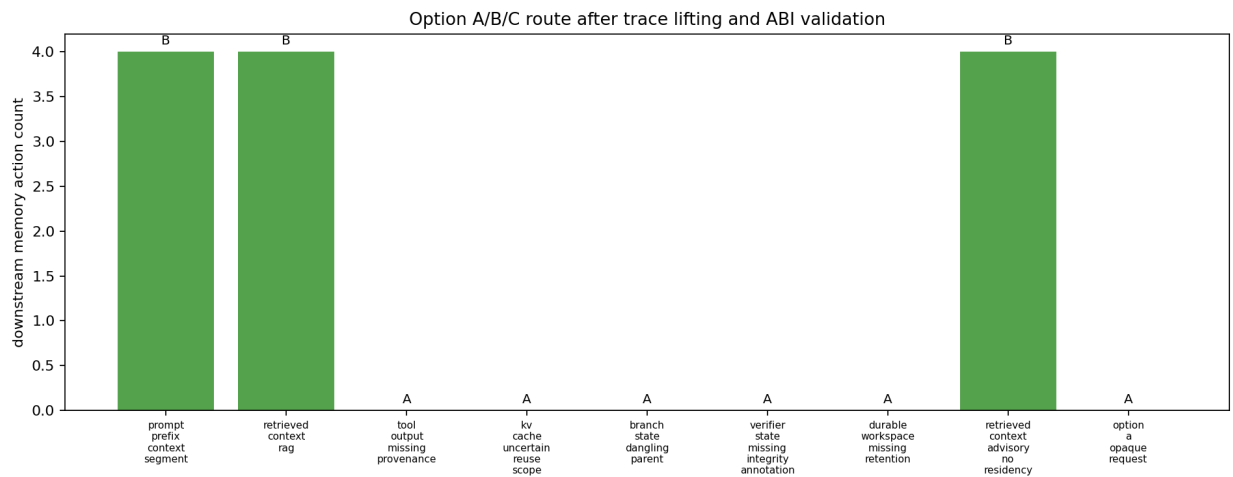


Figure 3: Option fallback results showing admitted Option B candidates, blocked candidates routed to Option A fallback, and opaque conventional requests.

Annotation class	Purpose
<code>reuse_scope_annotation</code>	Completes reuse scope when trace evidence cannot decide whether reuse is tenant-local or broader.
<code>tenant_security_annotation</code>	Completes tenant and security labels without allowing tenant switches or confidentiality downgrades.
<code>tool_provenance_annotation</code>	Binds tool outputs to observed tool-run lineage and freshness sources.
<code>branch_dependency_annotation</code>	Resolves branch IDs and parent object dependencies for trajectory/DAG state.
<code>verifier_integrity_annotation</code>	Names the observed target checked by verifier state.
<code>compression_safety_annotation</code>	Records whether compression is allowed for correctness-critical state.
<code>retention_policy_annotation</code>	Names a durable-retention policy for workspace artifacts.
<code>evidence_label_annotation</code>	Keeps evidence labels inside the allowed internal/runtime boundary and blocks production relabeling.

The merge rule is deterministic. Trace-derived fields form the base candidate. An annotation may fill only the fields allowed by its annotation class. The completed candidate is then rerun through `scripts.validate_memory_object_abi.classify`, the existing ABI validator. If the annotation conflicts with trace context or the completed object still violates the ABI, the candidate fails closed.

The main output, `data/annotation_merge_results.csv`, contains 17 rows: 6 ABI-admissible rows, 10 fail-closed rows, and 1 Option A opaque fallback. The supplied audit report also records option routing after merge: Option B = 3, Option C = 3, Option A = 11. Rejected, conflicting, and partial annotations emitted zero placement, reuse, compression, migration, or retention actions.

The fail-closed cases cover the expected unsafe paths: reuse-scope widening, tenant or security scope widening, provenance mismatch, unresolved branch parent, verifier target mismatch, lossy compression on correctness-critical state, missing durable-retention policy, missing mandatory reuse scope after annotation, and attempted `production_target` label upgrade.

The option-boundary output confirms that annotations can enable selected Option B and Option C paths, but only after validation. They do not make production claims. All production calibration, readiness, threshold success, causal validity, and claim-credit fields remained false.

The cycle 45 auditor reported no critical or moderate findings. The auditor validated the schema, examples, deterministic merge, ABI validator reuse, ABI integration helper accounting, conflict outputs, Option A fallback, zero production-credit fields, and figures. The validation event was `e9099ac3-92e6-47fe-a73c-dd0d5bff10f9`.

Discussion

Cycles 44-45 answer a practical architecture question: how can an existing serving stack move toward a memory-object-aware runtime without requiring all applications to produce

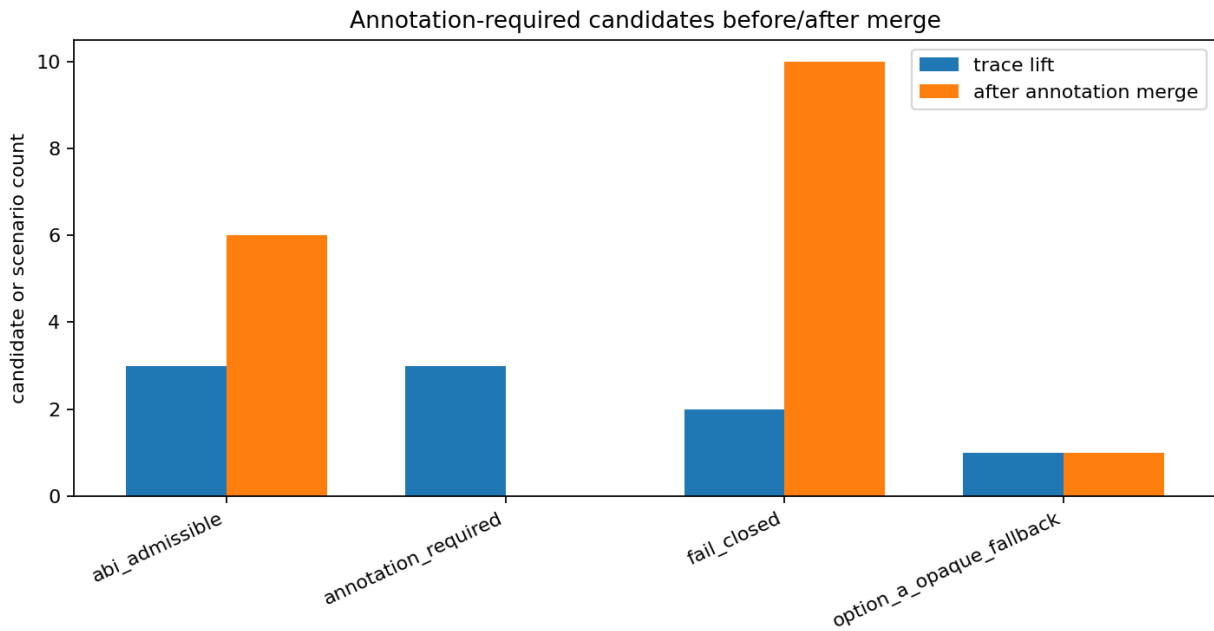


Figure 4: Annotation merge status showing six ABI-admissible completions, ten fail-closed cases, and one Option A opaque fallback.

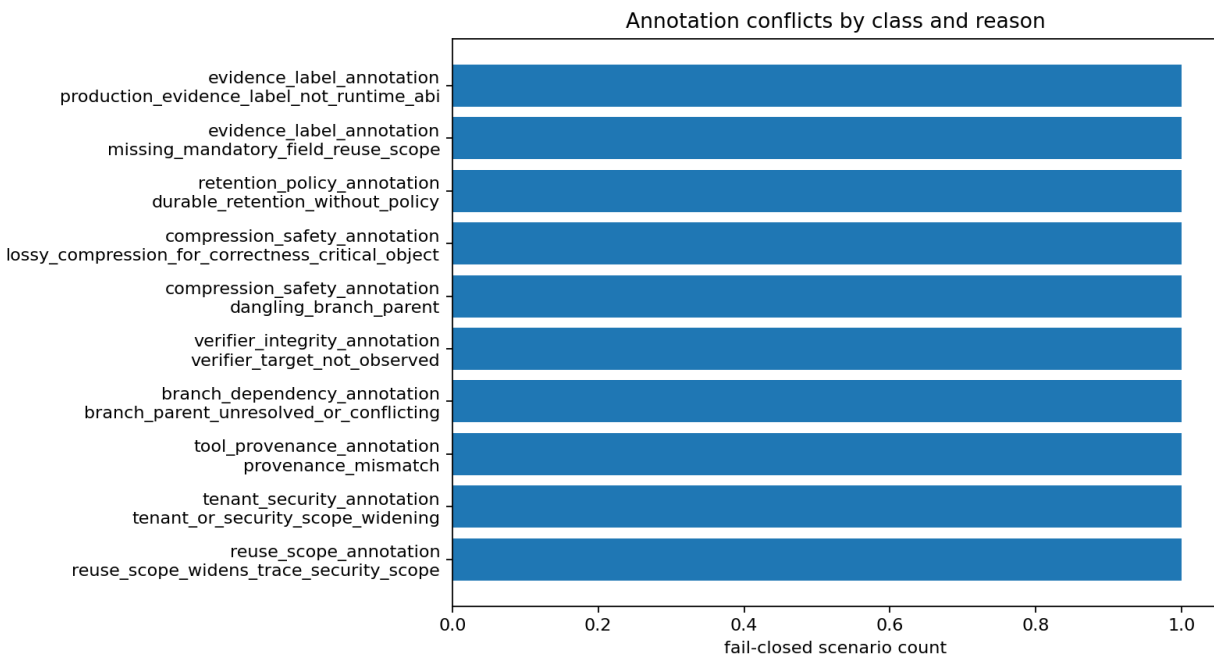


Figure 5: Annotation conflict failures showing each unsafe completion path failing closed with zero downstream memory actions.

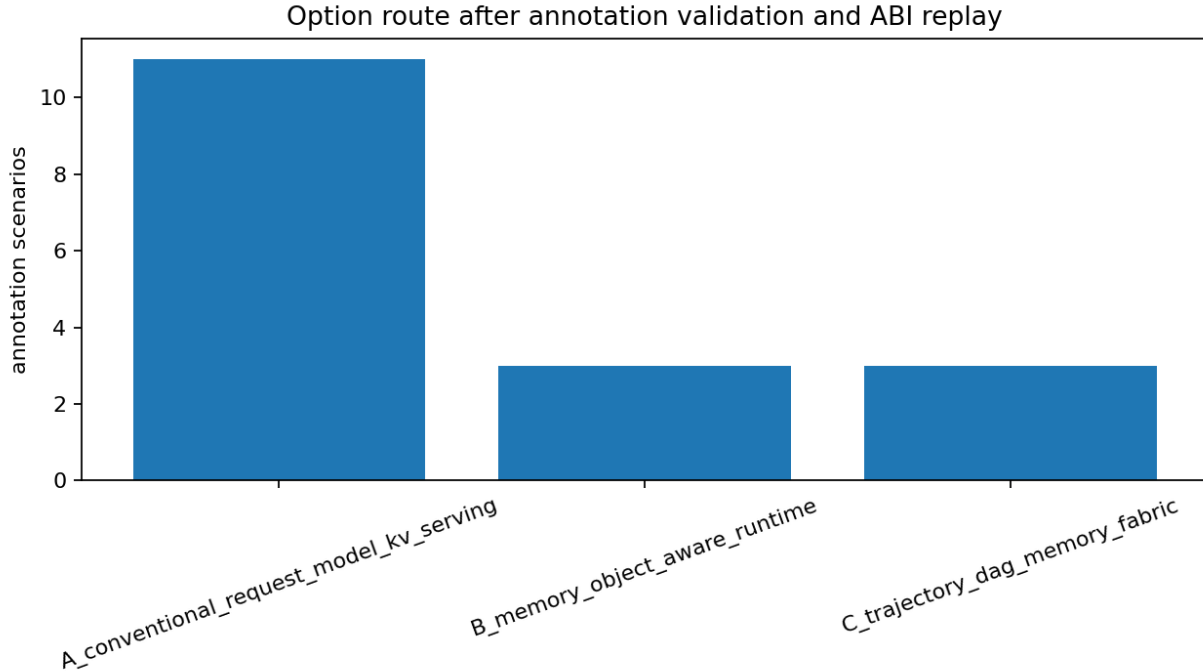


Figure 6: Annotation option boundary showing valid completions routed to Option B/C while invalid or opaque cases remain Option A with no claim credit.

perfect ABI objects on day one?

The answer developed here has three layers.

First, trace lifting provides a low-friction migration path. Existing request, runtime, and planner artifacts can produce candidate ABI objects when stable IDs, lineage, security labels, and planner context are available.

Second, annotation requirements make missing information visible. When trace data cannot infer reuse scope, provenance, branch dependency, verifier target, or durable retention policy, the system records that as `annotation_required` rather than defaulting to an unsafe value.

Third, constrained annotations complete only what they are allowed to complete. A valid annotation can turn selected KV, retrieved-context, tool-output, branch, verifier, or durable-workspace cases into ABI-admissible contracts. But the annotation cannot widen security scope, invent provenance, force unsafe compression, bypass retention policy, or upgrade internal evidence into production evidence.

This strengthens the memory-centric architecture proposal because it turns the ABI from a clean-room interface into a migration path. Existing systems can remain Option A where they are opaque, move to Option B where memory objects are safely exposed, and move to Option C where branch and verifier state are explicit enough to support trajectory-aware planning.

The production status is unchanged. These cycles are control-plane and migration evidence, not production telemetry. The current workspace still lacks real `production_target` material with linked gate evidence artifacts, complete source material, deployment-root integration, fresh lifecycle revalidation, and production-grade annotation provenance. Therefore no DC-001/DC-002 or Option B/C production-readiness claim is granted.

Open Questions

1. Can real serving stacks emit enough stable IDs, lineage, tenant/security labels, and planner context for trace lifting to cover a large share of agentic memory objects?
2. Which annotation classes can be generated automatically by compilers, frameworks, or runtimes, and which require developer intent?
3. What is the operational cost of maintaining correct annotations for branch state, verifier targets, and durable workspace artifacts over long-running agent trajectories?
4. How often do annotations conflict with observed trace or security context in real workloads?
5. Can production-grade annotation provenance be tied into the existing production evidence chain, including root enrollment, trust policy, attestation, intake custody, time-base integrity, redaction integrity, uncertainty, causal attribution, gate evidence artifacts, live collection, production replay, and claim expiry?
6. Does the Option B/C benefit survive when annotation generation, validation, and rejected-action overheads are measured on real production workloads?

References

- [1] NVIDIA, “NVIDIA H100 Tensor Core GPU,” NVIDIA Data Center, accessed 2026-05-11. <https://www.nvidia.com/en-us/data-center/h100/>
- [2] NVIDIA, “NVIDIA H200 Tensor Core GPU,” NVIDIA Data Center, accessed 2026-05-11. <https://www.nvidia.com/en-us/data-center/h200/>
- [3] NVIDIA, “NVIDIA DGX B200: The Foundation for Your AI Factory,” NVIDIA Data Center, accessed 2026-05-11. <https://www.nvidia.com/en-gb/data-center/dgx-b200/>
- [4] NVIDIA, “Introduction to NVIDIA DGX H100/H200 Systems,” NVIDIA DGX H100/H200 User Guide, accessed 2026-05-11. <https://docs.nvidia.com/dgx/dgxm100-user-guide/introduction-to-dgxm100.html>
- [5] PCI-SIG, “PCI Express 6.0 Specification,” PCI-SIG, accessed 2026-05-11. <https://pcisig.com/pci-express-6.0-specification>
- [6] NVM Express, “Specifications,” NVM Express, accessed 2026-05-11. <https://nvmexpress.org/specifications/>
- [7] Compute Express Link Consortium, “CXL Specification,” Compute Express Link, accessed 2026-05-11. <https://computeexpresslink.org/cxl-specification/>
- [8] MLCommons, “MLPerf Inference: Datacenter Benchmark,” MLCommons, accessed 2026-05-11. <https://mlperf.org/benchmarks/inference-datacenter/index.html>
- [9] Woosuk Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” arXiv, 2023. <https://arxiv.org/abs/2309.06180>
- [10] Intel, “Intel Xeon 6 Processors with MRDIMM — Solution Brief,” Intel, accessed 2026-05-11. <https://www.intel.com/content/www/us/en/content-details/919018/intel-xeon-6-processors-with-mrdimm-solution-brief.html>
- [11] AMD, “AMD EPYC 9005 Processor Architecture Overview,” AMD, accessed 2026-05-11. https://www.amd.com/content/dam/amd/en/documents/epyc-technical-docs/user-guides/58462_amd-epyc-9005-tg-architecture-overview.pdf
- [12] vLLM Project, “Automatic Prefix Caching,” vLLM Documentation, accessed 2026-05-11. https://docs.vllm.ai/en/latest/design/prefix_caching/

[13] Sajal Regmi and Chetan Phakami Pun, “GPT Semantic Cache: Reducing LLM Costs and Latency via Semantic Embedding Caching,” arXiv, 2024. <https://arxiv.org/abs/2411.05276>

[14] CXL Consortium, “CXL Consortium Releases Compute Express Link 2.0 Specification,” Business Wire, 2020. <https://www.businesswire.com/news/home/20201110005037/en/CXL-Consortium-Releases-Compute-Express-Link-2.0-Specification>

Appendix: Implementation Details

Code Organization

Cycle 44 added or used the trace-to-ABI migration files:

File	Lines	Purpose
scripts/lift_trace_to_memory_object_abi.py	373	Lifts conventional trace/runtime/planner records into ABI candidates and status outputs.
scripts/plot_trace_to_abi_lift.py	113	Renders trace-lift status, missing-field, and option-fallback figures.
tests/verify_trace_to_abi_lift.py	135	Verifies trace lifting, ABI validator reuse, Option A fallback, action gating, and figures.
memory-centric-agent/trace_to_abi_migration.md	32	Documents the migration path and fail-closed boundary.

Cycle 45 added or used the annotation files:

File	Lines	Purpose
scripts/build_memory_object_annotations.py	215	Builds annotation schema, examples, and requirement fixtures.
scripts/merge_trace_annotations_to_abi.py	238	Merges annotations into trace-derived candidates and reruns ABI validation and integration accounting.
scripts/plot_memory_object_annotations.py	84	Renders annotation status, conflict, and option-boundary figures.
tests/verify_memory_object_annotations.py	162	Verifies annotation schema/examples, merge constraints, ABI validation reuse, action gating, and figures.

File	Lines	Purpose
memory-centric-agentive/memory_object_annotation_contract.md	31	Documents annotation classes, constraints, merge rules, and production boundary.

Data Outputs

File	Rows	Meaning
data/trace_to_abi_lift_results.csv	9	Legacy trace-lift outcomes by case.
data/trace_to_abi_missing_fields.csv	7	Missing or ambiguous fields that block or qualify lifting.
data/trace_to_abi_option_fallbacks.csv	9	Option routing and fallback behavior after trace lifting.
data/trace_to_abi_annotation_requirements.csv	3	Annotation requirements surfaced by trace lifting.
data/memory_object_annotation_schema.csv	8	Annotation classes and allowed field completions.
data/memory_object_annotation_examples.jsonl	17	Valid and invalid annotation examples.
data/memory_object_annotation_requirements.csv	5	Required annotation-completion cases.
data/annotation_merge_results.csv	17	Merge outcomes after applying annotations.
data/annotation_completed_abi_candidates.jsonl	6	Completed candidates that became ABI-admissible.
data/annotation_conflict_failures.csv	10	Unsafe or incomplete annotations that failed closed.
data/annotation_integration_results.csv	17	Annotation outputs replayed through ABI/integration accounting.
data/annotation_option_boundary.csv	17	Option routing and zero-credit boundary rows.

Figure Inventory

Figure	Dimensions	Placement
data/trace_to_abi_status_counts.png	1760 x 768	Cycle 44 trace-lift outcomes.

Figure	Dimensions	Placement
data/trace_to_abi_missing_fields.png	1760 x 768	Cycle 44 missing-field and annotation-required evidence.
data/trace_to_abi_option_fallbacks.png	1920 x 768	Cycle 44 Option A/B/C routing boundary.
data/memory_object_annotation_status.png	1440 x 768	Cycle 45 annotation merge outcomes.
data/memory_object_annotation_conflicts.png	1600 x 880	Cycle 45 fail-closed annotation conflicts.
data/memory_object_annotation_option_boundary.png	1280 x 736	Cycle 45 option and claim-credit boundary.

Validation Results

Cycle 44 M-TRACEABI-1 was auditor-validated. The auditor reported that candidate generation, ABI validator reuse, fallback and missing-field outputs, migration documentation, figures, adjacent ABI/integration/control-plane regressions, `promise_check`, and `org_check` were checked. A scoped auditor patch tied lifted admissible-candidate action counts to the existing ABI integration helper path. Incomplete or ambiguous trace-derived contracts emitted zero downstream memory actions.

Cycle 45 M-ANNOT-1 was auditor-validated with no critical or moderate findings. The supplied audit report records:

- `python3 tests/verify_memory_object_annotations.py` passed.
- Syntax compile passed.
- Adjacent regressions passed: trace-to-ABI lift, memory-object ABI, ABI integration, and architecture control-plane progression.
- `promise_check` was green with 230 events and 43 plan milestones.
- `org_check` exited 0 with known root package warnings only.
- Annotation figures were nonblank.
- `annotation_merge_results.csv` had 17 rows.
- 6 rows were `abi_admissible`, 10 were `fail_closed`, and 1 was `option_a_opaque_fallback`.
- Option routing after annotation was B = 3, C = 3, A = 11.
- Rejected, conflicting, or partial annotations emitted zero downstream placement, reuse, compression, migration, or retention actions.
- All production calibration, readiness, threshold, causal-validity, and claim-credit fields remained false.

The only minor issue was the known `org_check` warning about root package artifacts: `CURATION.yaml`, `memory_centric_agentic_inference_package_2026-05-12T0019.zip`, `memory_centric_agentic_inference_package_2026-05-12T1804.zip`, and `memory_centric_agentic_inference_package_latest.zip`.

Source Session References

Cycle	Role	Session ID
44	researcher	15321c2d-5734-4434-803e-bf9a4b7a5904
44	worker	5d05a8ba-a51c-460d-a13e-746f59f6584d

Cycle	Role	Session ID
44	auditor	0c742462-3c65-4490-815c-f53ea7d9aec3
45	researcher	039107f3-6458-461e-ab7f-c10b11797eed
45	worker	d6346464-fa09-46ff-a4a7-ee8e082f89db
45	auditor	f0587bb3-2d2d-4b04-be7c-ded4019d58f0

Cross-Reference Map

Origin	Consuming artifact	Flow
data/runtime_policy_decisions.csv and data/memory_plan_actions.csv	data/trace_to_abi_lift_results.csv, data/trace_to_abi_option_fallbacks.csv	Existing runtime/planner artifacts provide migration source context for trace-derived ABI candidates.
data/trace_to_abi_lift_results.csv	data/trace_to_abi_annotation_requirements.csv, data/annotation_merge_results.csv	Trace lifting identifies which candidates are already admissible, which fail closed, and which require constrained annotations.
data/memory_object_annotation_examples.jsonl	data/annotation_merge_results.csv, data/annotation_conflict_failures.csv	Annotation examples either complete ABI candidates or fail closed under conflict rules.
data/annotation_merge_results.csv	data/annotation_integration_results.csv, data/annotation_option_boundary.csv	Completed candidates are rerun through ABI validation and integration action gating before Option B/C routing.
data/production_target_replay_claim_boundary.csv	data/claim_expiry_claim_boundary.csv, data/annotation_option_boundary.csv	Existing production-claim boundaries remain upstream constraints; annotation evidence does not grant production credit.

Manifest Snapshot

MANIFEST.md was updated for cycles 44-45 while preserving its existing ## Key Files section. The snapshot now records 99 Python scripts in scripts/, 4 Wolfram scripts, 30 test scripts, 1 tool script, 24,344 total scripts/ lines, 44 top-level markdown model/synthesis files under memory-centric-agentic/, 250 CSV data/model files, 113 figures under data/, and 39 completed, assessed, or designed sub-topics.

The manifest now includes M-TRACEABI-1 and M-ANNOT-1 in the validation snapshot and cross-reference map. It also states the current production boundary: trace-lift fixtures and annotation fixtures remain non-production evidence, and real production_target telemetry plus production-grade annotation provenance are still required before any DC-001/DC-002 or Option B/C claim can become production-ready.